

In the movie Die Hard 3, the heroes must obtain exactly 4 gallons of water using a 5 gallon jug, a 3 gallon jug, and a water faucet. Our goal: to get *TLC* to solve the problem for us.

First, we write a spec that describes all allowable behaviors of our heroes.

EXTENDS *Naturals*

This statement imports the definitions of the ordinary operators on natural numbers, such as +.

We next declare the specification's variables.

VARIABLES *big*, The number of gallons of water in the 5 gallon jug.
 small The number of gallons of water in the 3 gallon jug.

We now define *TypeOK* to be the type invariant, asserting that the value of each variable is an element of the appropriate set. A type invariant like this is not part of the specification, but it's generally a good idea to include it because it helps the reader understand the spec. Moreover, having *TLC* check that it is an invariant of the spec catches errors that, in a typed language, are caught by type checking.

Note: TLA+ uses the convention that a list of formulas bulleted by $\wedge$ or $\vee$ denotes the conjunction or disjunction of those formulas. Indentation of subitems is significant, allowing one to eliminate lots of parentheses. This makes a large formula much easier to read. However, it does mean that you have to be careful with your indentation.

$$\begin{aligned} \textit{TypeOK} &\triangleq \wedge \textit{small} \in 0 \dots 3 \\ &\quad \wedge \textit{big} \in 0 \dots 5 \end{aligned}$$

Now we define of the initial predicate, that specifies the initial values of the variables. I like to name this predicate *Init*, but the name doesn't matter.

$$\begin{aligned} \textit{Init} &\triangleq \wedge \textit{big} = 0 \\ &\quad \wedge \textit{small} = 0 \end{aligned}$$

Now we define the actions that our hero can perform. There are three things they can do:

- Pour water from the faucet into a jug.
- Pour water from a jug onto the ground.
- Pour water from one jug into another

We now consider the first two. Since the jugs are not calibrated, partially filling or partially emptying a jug accomplishes nothing. So, the first two possibilities yield the following four possible actions.

$$\begin{aligned} \textit{FillSmallJug} &\triangleq \wedge \textit{small}' = 3 \\ &\quad \wedge \textit{big}' = \textit{big} \\ \textit{FillBigJug} &\triangleq \wedge \textit{big}' = 5 \\ &\quad \wedge \textit{small}' = \textit{small} \\ \textit{EmptySmallJug} &\triangleq \wedge \textit{small}' = 0 \\ &\quad \wedge \textit{big}' = \textit{big} \end{aligned}$$

$$\begin{aligned} \text{EmptyBigJug} \quad \triangleq \quad & \wedge \text{big}' = 0 \\ & \wedge \text{small}' = \text{small} \end{aligned}$$

We now consider pouring water from one jug into another. Again, since the jugs are not calibrated, when pouring from jug A to jug B , it makes sense only to either fill B or empty A . And there's no point in emptying A if this will cause B to overflow, since that could be accomplished by the two actions of first filling B and then emptying A . So, pouring water from A to B leaves B with the lesser of (i) the water contained in both jugs and (ii) the volume of B . To express this mathematically, we first define $\text{Min}(m, n)$ to equal the minimum of the numbers m and n .

$$\text{Min}(m, n) \triangleq \text{IF } m < n \text{ THEN } m \text{ ELSE } n$$

Now we define the last two pouring actions. From the observation above, these definitions should be clear.

$$\begin{aligned} \text{SmallToBig} \quad \triangleq \quad & \wedge \text{big}' = \text{Min}(\text{big} + \text{small}, 5) \\ & \wedge \text{small}' = \text{small} - (\text{big}' - \text{big}) \\ \text{BigToSmall} \quad \triangleq \quad & \wedge \text{small}' = \text{Min}(\text{big} + \text{small}, 3) \\ & \wedge \text{big}' = \text{big} - (\text{small}' - \text{small}) \end{aligned}$$

We define the next-state relation, which I like to call Next . A Next step is a step of one of the six actions defined above. Hence, Next is the disjunction of those actions.

$$\begin{aligned} \text{Next} \quad \triangleq \quad & \vee \text{FillSmallJug} \\ & \vee \text{FillBigJug} \\ & \vee \text{EmptySmallJug} \\ & \vee \text{EmptyBigJug} \\ & \vee \text{SmallToBig} \\ & \vee \text{BigToSmall} \end{aligned}$$

We define the formula Spec to be the complete specification, asserting of a behavior that it begins in a state satisfying Init , and that every step either satisfies Next or else leaves the pair $\langle \text{big}, \text{small} \rangle$ unchanged.

$$\text{Spec} \triangleq \text{Init} \wedge \Box[\text{Next}]_{\langle \text{big}, \text{small} \rangle}$$

Remember that our heroes must measure out 4 gallons of water. Obviously, those 4 gallons must be in the 5 gallon jug. So, they have solved their problem when they reach a state with $\text{big} = 4$. So, we define NotSolved to be the predicate asserting that $\text{big} \neq 4$.

$$\text{NotSolved} \triangleq \text{big} \neq 4$$

We find a solution by having TLC check if NotSolved is an invariant, which will cause it to print out an “error trace” consisting of a behavior ending in a states where NotSolved is false. Such a behavior is the desired solution. (Because TLC uses a breadth-first search, it will find the shortest solution.)